Project 1. Socket programming practice

In this project, we build a pair of simple network applications, S-client and S-server, using the standard Berkeley socket API on Linux. We use the TCP socket API (socket(), bind(), listen(), connect(), accept(), write(), read(), etc.) for this programming assignment. For socket programming APIs, please refer to Beej's guide to network programming.

A. S-client

`s-client` receives the input message from standard input (`stdin`) and sends a request that contains the input message to `s-server`. Then, it receives the response from `s-server` and writes it to standard output (`stdout`). After this, `s-client` stops. One can run `s-client` as follows.

```
./s-client -p 1080 -s 147.256.38.7 < file-to-send.txt
```

(1) The input message can be any content whose size can be up to 10 MB (= 10×1024×1024 bytes). If the input message exceeds 10 MB, `s-client` takes it up to 10 MB and discards the rest. Any message smaller than 10 MB should be terminated by EOF. If the input size is 0 (i.e, the user enters `EOF` without any content), then `s-client` prints out an error message and stops.

(2) The request format of `s-client` is (similar to HTTP protocol message format) as follows.

```
POST message SIMPLE/1.0
Host: [s-server-domain-name]
Content-Length: [byte-count]
(empty line)
[message-content]
```

where each line must end with two characters, `'\r'` (a carriage return, CR) and `'\n'` (a line feed, LF), represented as CRLF. [s-server-domain-name] is the domain name of the server (e.g., www.nytimes.com) and [byte-count] is the number of bytes of the input message. s-server-domain-name can be "localhost" or an IP address like "143.247.3.1" and for this assignment, the server does not need to validate it – it's OK as long as there is some string in the host header field.

(empty line) has no content in the line – that is, the '`Content-Length`' line ends with CRLFCRLF. Except for the first line (i.e., the `POST` line), request header lines (e.g., italicized part such as *`Host:`* and *`Content-Length:`*) are *case-insensitive* (e.g., "`hOsT:`" is the same as "`Host:`") and there can be any number of whitespaces (checked by `isspace()`) before and after a value (e.g., [s-server-domain-name] and [byte-content]). Similarly, there can be any number of whitespaces around, "`message`", "`SIMPLE/1.0`" (but no whitespaces before "`POST`", "`Host:`" and "`Content-Length`"). There is no specific order requirement between "`Host:`" and "`Content-Length:`". As long as both header fields exist, then the

request is considered well-formed. Please refer to the [HTTP RFC 1945 (HTTP/1.0)] for the request header line format.

(3) After `s-client` sends the request, it should receive the response from `s-server`. The response format is as follows.

```
SIMPLE/1.0 200 OK
Content-Length: [byte-count]
(empty line)
[message-content]
```

The same spacing rules apply: no whitespaces before the first token (`"SIMPLE/1.0"`, `"Content-Length")` with any number of whitespaces around other tokens. `"200 OK"` means that the request was well-formed and the server sends a normal response. However, the server may respond with an error response (see below) if the client-sent request is malformed.

(4) While receiving the response, `s-client` should write only the response body (only the `[message-content]` part) to standard output if the response starts with "`200 OK`". For an error response (which has only a response header), the client should write the entire response header to standard output.

B. S-server

`s-server` waits for and handles connections from `s-client`. To run `s-server` with the listening port of 1080, the command should look like

```
./s-server -p 1080
```

(1) When `s-server` starts, it listens on the specified TCP port (between 1024 and 65535), and waits for connections from one or more `s-client` instances. When a TCP connection is established, it receives a request from `s-client`, and if the request is well-formed, it sends a response that looks like

```
SIMPLE/1.0 200 OK
Content-Length: [byte-count]
(empty line)
[message-sent-from-s-client]
```

where `[byte-count]` is the same as in the `"Content-Length:"` field in the request. And `[message-sent-from-s-client]` should be the same content in the request body received from `s-client`. (This should be your implementation, but TA's `s-server` for grading may send a response body with a different content than the original one from `s-client`. Nevertheless, the content length of the response body will be the same as in the request.) `s-client` should work well as long as the content length of the response body is the same as the original content. After sending the response, `s-server` closes the connection.

(**WARNING**) The server must receive the entire message from `s-client` first before sending out the response. Otherwise, `s-client` and `s-server` could get stuck (or deadlocked) unless you use non-blocking socket system calls explained below (e.g., both `s-client` and `s-server` would block at sending the data but neither would receive the data, so no progress!).

(2) `s-server` should respond with an error if the request is ill-formed. There could be many error cases - the first line of the request could be malformed or one of the request headers (`content-length:` or `host:`) is missing, etc. For all such cases, `s-server` responds with the following response and closes the connection.

```
SIMPLE/1.0 400 Bad Request
(empty line)
```

(**WARNING**) It's important that `s-client` should send the exact same amount of data as specified in the `content-length` field. Otherwise, the connection could get stuck – e.g., `s-client` sends a smaller amount of data than promised in the header field and waits for a response from `s-server` while `s-server` still waits for more data to arrive. Technically, you can implement a timeout mechanism to avoid such a situation, but we don't implement it for this assignment.

(3) `s-server` should handle multiple simultaneous connections from `s-client` instances. Even if a connection *isn't* finished (e.g., `s-server` might spend lots of time on receiving a request from a very slow `s-client` instance), `s-server` should be able to handle other active connections. You are supposed to support ***at least 5 concurrent connections*** (e.g., it's OK for the 6th connection to wait until one of the five connections finishes).

Implementation hints:

- Be meticulous about error processing - your program should do proper error processing (e.g., content-length is 0, negative or larger than the maximum value. the sent data size is not the same as the content-length, etc.) In case of an error, print out an error message to stderr, and if the program can't continue, stop the program. The decision is up to you.

- More elaboration on the message size: it is ok if a server or a client does not detect the case where the message size exceeds the content-length. If a client detects that the message size is smaller than the content-length, it should print out an error. It is OK for a server to get stuck if the client-sent message has a smaller size than promised in the content-length of the request.

- Do not assume that the input message is always a text message. It can contain arbitrary content that includes valid "zeros" in the binary content. This means that you should be careful when using string functions (e.g., `strlen()`, `strstr()`) on the input message.

- The number of bytes received at a time could be arbitrary. It could be as small as one byte. Also, even if the other endpoint sends 32KB of data with one write(), you can receive 1KB of the data at a time at each `read()` call for 32 times. Do not assume that `read()`/`write()` returns the same value as the last argument (count) to the system call.

- Avoid gross inefficiency in reading or writing the data. For example, reading/writing one byte (or a small number of bytes) at a time would be inefficient, so avoid that.

- Assume the maximum header length is 1024 bytes. You can print out an error message and stop if the header is larger than 1024 bytes.

- `SIGPIPE` is generated if you call `write()` on a connection that's already closed by the other endpoint. Your application would get killed if it does not handle the signal. We recommend calling `signal(SIGPIPE, SIG_IGN);` early on to ignore the signal.

Handling concurrent connections:

There are largely two ways to support multiple simultaneous connections.

a. One approach is to use `fork()` right after calling `listen()`, which is called the "prefork" approach. Then, the server's listening socket (with the same connection listen queue) will be shared by `fork()`'ed processes. After `fork()`, a child process typically calls `accept()` that "blocks" for a new connection. Note that a system call is said to "block" if it does not return to the caller (or to the user-level code) until it finishes its operation or an error occurs. By default, all socket system calls are blocking - `read()` won't return until some message arrives from the other endpoint or the other endpoint closes the connection (then, `read()` returns 0), etc. If multiple connections are established concurrently, the Linux kernel would distribute the new connections to multiple processes that simultaneously block at `accept()`. The prefork approach is the simplest way to support multiple concurrent connections.
b. Another approach is to use "non-blocking" system calls. If you set a socket as non-blocking (e.g., `fcntl(sock, F_SETFL, fcntl(sock, F_GETFL, 0) | O_NONBLOCK, 0);`), then subsequent system calls with the socket would not block. For example, `read()` would not wait and return immediately even if there is nothing to read. The same applies to `accept()` - it would return -1 (with `errno == EAGAIN`) if there's no established connection to accept. This way, your program can handle multiple concurrent connections (represented as sock1, sock2,... sockn) without blocking at one of the connections. Of course, blindly calling `read()` without knowing there is any data to read would significantly waste CPU cycles. So, there is an event notification system call (e.g., `epoll_wait()`, `select()` in Linux and `kqueue()` in BSD, etc.) that *blocks* until some event(s) with any of the monitored sockets arise(s). You can check which socket has what event(s) (read or write event) and invoke relevant system calls to handle the event(s). This way of handling concurrent connections is called event-driven programming, and it's known to be

more efficient than fork()-based (or thread-based) approach although it's more difficult to write the code.

c. Typical real-world, high-performance servers adopt the hybrid approach. They prefork (or create multiple threads) and then use event-driven programming within one thread/process.

We suggest doing (a) if you are unfamiliar with the event-driven architecture. There is no requirement to implement the server using the event-driven architecture for Assignment 1. However, the server for Assignment 2 must use the event-driven architecture to receive full credit.

Collaboration Policy:

- You can discuss with other students, but you should write the code on your own – you should **NOT** look at someone else' code nor show/give your code to anyone else. You should NOT consult generative AIs for doing the homework – writing or debugging your code, etc. You can view and/or study generic network socket programming code or the code that explains the usage of system/socket calls.
- Please stay away from any kind of plagiarism. If you are not sure about anything, ask TAs and the instructor.

Submission:

- We provide `macro.h`, and `Makefile`. Feel free to use `macro.h` or not. Do **not** modify either file when you submit them.
- Submit `sclient.c, sserver.c, readme.pdf, macro.h` and `Makefile` in a gzipped tar file format. Do **NOT** submit any other files. Write (1) your detailed implementation strategy, especially on how your server supports concurrent connections, (2) how you tested your programs, (3) any bugs in the current code, (4) all collaborators in `readme.pdf`. The gzipped tar file should be "{YourID_without_dash}_{YourNameInEnglish}_assign1". If your student ID is 2024-12345 and your name is "Haechan Kim", then it should be `202412345_HaechanKim_assign1`. **Note that you should use your student ID without the dash (-)**. Assume you have the files in the current directory:

```
$ mkdir 202412345_HaechanKim_assign1
$ cp sclient.c sserver.c macro.h readme.pdf Makefile
\ 202412345_HaechanKim_assign1
$ tar zcf 202412345_HaechanKim_assign1.tar.gz
\ 202412345_HaechanKim_assign1
```

Alternatively, you may use the given script to make the tar file automatically.

```
$ ./submit.sh 202412345 HaechanKim
```

Late Policy:

- We will not accept late submission. Any late submission will result in zero credit. We may consider extension if you are in extraordinary circumstances (e.g., hospitalization). Please inform us as soon as you know

Grading:

- Integrity of the submission file - 5 pts
  - Submission file format, all required files, no modification of macro.h/Makefile, no additional files, etc)
- Content of `readme.pdf` - 10 pts
  - All required information
- Coding style and compilation issues - 5 pts
  - Proper indentation and comments, proper variable naming, remove unnecessary global variables, no compilation warnings in, etc.
  - Please note that you will get zero credit if compilation fails. We will not allow you to fix any errors in your code once it's submitted.
- Correctness of the client - 30 pts
- Correctness of the server - 40 pts
- Error processing - 10 pts
- If your code does not compile with our Makefile, you'll get zero credit. However, if it is a simple error (e.g., forgot to include a header file, uses a custom macro defined by you in your macro.h, etc.) that can be easily fixed, we can still grade it by fixing the error. In this case, you will get 10% of deduction as penalty on your earned points. So, be careful!